НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Звіт до комп'ютерного практикуму №5
З дисципліни «Комп'ютерне моделювання дискретно-подійних систем»
«Формалізація дискретно-подійних систем стохастичною мережею Петрі»

Прийняв:
Викладач
Коринецький Дмитро Сергійович
«12» листопада 2026 р.

Виконав:
Студент 4 курсу, гр. ІП-34
Скачков Іван
Олексійович

2026 р.

Зміст

Лабораторна робота №5 3
Технічне завдання 4
Виконання 6
Підхід до формалізації і моделювання 6
Лістинг допоміжного класу 7
Завдання 1 9
Мережа Петрі 9
Лістинг файлів 10
Результат роботи 10
Висновок 10
Завдання 2 12
Формалізація 12
Лістинг файлів 13
Результат роботи 14
Висновок 15
Завдання 3 16
Формалізація 16
Лістинг файлів 17
Результат роботи 18
Висновок 19
Завдання 4 20
Мережа Петрі 20
Лістинг файлів 21
Результат роботи 22
Висновок 22
Висновок 23
Посилання 24

Лабораторна робота №5

Тема: Формалізація дискретно-подійних систем стохастичною мережею Петрі.

Мета: навчитися формалізувати процеси функціонування дискретно-подійних систем стохастичними мережами Петрі з часовими затримками, конфліктними, багатоканальними переходами та інформаційними зв’язками і визначати характеристики систем імітацією побудованих мереж.

Засоби: графічний редактор і бібліотека PetriObjModelPaint 2.8.2 (І. В. Стеценко) — мережі побудовано в редакторі і збережено у форматах PNML і Java-методу; експерименти виконано програмами на Java 25 з алгоритмом імітації цієї бібліотеки (класи PetriSim і PetriObjModel).

Технічне завдання

Розробити мережі Петрі для таких задач.

1. Конвеєрна система складається з п’ятьох обслуговуючих пристроїв, розташованих уздовж стрічки конвеєра. Деталі надходять на перший пристрій із постійною швидкістю 4 одиниці за 1 хвилину. Тривалість обслуговування на кожному пристрої розподілена за експоненціальним законом з математичним сподіванням 1 хвилина. Вільного місця перед кожним пристроєм немає, тому пристрій може зняти деталь із конвеєра, тільки якщо знаходиться в стані «вільний». Якщо перший пристрій вільний, деталь обробляється на ньому і по закінченні обробки залишає систему. Якщо перший пристрій зайнятий у момент надходження деталі, деталь по конвеєру надходить до другого пристрою; інтервал проходження деталі між пристроями — 1 хвилина. Якщо при прямуванні деталі по конвеєру всі пристрої були зайняті, вона повертається до першого пристрою з затримкою 5 хвилин. Метою моделювання є визначення статистичних характеристик часу перебування деталі в системі, завантаження обслуговуючих пристроїв і кількості зайнятих пристроїв.
2. Експериментальна роботизована гнучка виробнича система має два верстати з числовим пультом керування, три роботи, пункт прибуття і склад оброблених деталей. Деталі прибувають на пункт прибуття кожні 40 секунд згідно з експоненціальним законом розподілу, захоплюються одним з вільних роботів і переміщуються ним до першого верстата, після чого робот звільняється. Після обробки на першому верстаті деталь захоплюється одним з роботів і переміщується на другий верстат, а після обробки на другому верстаті — одним з роботів на склад. Кожний з верстатів може одночасно обробляти до трьох деталей. Час переміщення робота між пунктом прибуття та першим верстатом, першим і другим верстатом, другим верстатом та складом — відповідно 6, 7 і 5 секунд незалежно від того, холостий це хід чи ні. Роботу потрібно 8±1 секунд на захоплення або вивільнення деталі. Час обробки на першому верстаті —

нормальний із середнім 60 с і стандартним відхиленням 10 с, на другому — експоненційний із середнім 100 с. Мета — визначити найкращий з точки зору пропускної здатності спосіб закріплення роботів: 1) по одному роботу на кожний із трьох шляхів переміщення деталей; 2) кожний робот може використовуватися на кожному шляху (займається найближчий з роботів).

3. На маршруті приміського сполучення працюють два мікроавтобуси A і B, кожний має n місць. Пасажир сідає в мікроавтобус B тільки у випадку, коли автобуса A немає. Мікроавтобус відправляється на маршрут, якщо всі місця в ньому зайняті. Пасажири підходять до зупинки через 0,5±0,2 хвилини і, якщо немає мікроавтобусів, утворюють чергу; якщо черга більша ніж 30 осіб, пасажир іде до іншого маршруту. На проходження маршруту мікроавтобус A витрачає 20±5 хвилин, B — 30±5 хвилин; після того як пасажири звільнили автобус (5±1 хвилин), він їде у зворотному напрямку тим же чином. Плата за проїзд — 20 гривень; стільки ж автопідприємство недоотримує, якщо пасажир обирає інший маршрут. Визначити час очікування пасажира в черзі, кількість місць n (не більше 25), при якій він мінімальний, і виручку за день, якщо мікроавтобуси працюють 10 годин на добу.
4. Час між надходженнями замовлень на холодильники має експоненціальний розподіл із середнім 0,2 тижня. Якщо холодильника в запасі немає, покупець у 80 % випадків іде в інший магазин (продаж не відбувся), а у 20 % робить повторне замовлення і чекає наступної партії. Запас проглядається кожні 4 тижні; якщо поточний стан запасу (наявний запас плюс замовлені раніше мінус невдоволений попит) не більший за 18 холодильників, розміщується замовлення, що доводить його до контрольного рівня 72. Час доставки постійний — 3 тижні. Початкові умови: запас 72 холодильники, невдоволеного попиту немає. Визначити середню кількість холодильників у запасі і середній час між продажами, що не відбулися.

Виконання

Підхід до формалізації і моделювання

Для кожної системи виділено події, що в ній відбуваються, і поставлено їм у відповідність переходи мережі Петрі; умови виникнення подій подано позиціями, а виконання умов — маркерами. Часові затримки задано в переходах законами розподілу з умови задачі, вибір однієї з кількох подій — пріоритетом або ймовірністю запуску переходу, необмежену кількість одночасних подій (деталі на конвеєрі, рейси) — багатоканальними переходами, а перевірку умови без її зміни — інформаційними дугами (на рисунках — пунктир).

Мережі побудовано в редакторі PetriObjModelPaint, перевірено його засобами (Validate, пробний запуск) і збережено в каталозі nets: зображення (.png), мережа у форматі PNML (.pnml) і Java-метод, що створює мережу (.txt, команда Save net as method). Ці методи зібрано у файлі Nets.java; параметри, які перебираються в експериментах (інтервал надходження, кількість місць, параметри стратегії запасів), винесено в аргументи методів. Імітацію виконує бібліотека: мережа загортається в Петрі-об’єкт PetriSim, модель PetriObjModel просуває час до найближчої події і накопичує середні за часом кількості маркерів у позиціях і в переходах (допоміжний клас Sim.java).

Середній час перебування об’єкта в системі визначено за формулою Літтла як відношення середньої кількості об’єктів у системі до інтенсивності їх виходу, а середній час очікування в черзі — як сумарне очікування $\Sigma M \cdot \Delta t$, поділене на кількість обслужених.

Під час налагодження виявлено особливість алгоритму бібліотеки: конфлікт розв’язується серед усіх активних переходів з однаковим пріоритетом, навіть якщо вони не мають спільних вхідних позицій, а перехід з імовірністю 1,0 отримує частку 1/k. Генератор вимог стає активним у той самий момент, коли з’являється нова вимога, і без додаткових заходів брав участь у розіграші, спотворюючи задані ймовірності (замість 0,8 виходило 0,93). Тому переходи, що

вибирають подію за ймовірністю, отримали пріоритет 1 — вищий за пріоритет генератора.

Лістинг допоміжного класу

Файл src\Sim.java:

```
import java.util.ArrayList;
import java.util.List;
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriObjModel;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriSim;
import ua.stetsenkoinna.petriobj.PetriT;
import ua.stetsenkoinna.petriobj.StateTime;


/**
 * Імітація мережі Петрі: Петрі-об'єкт PetriSim у моделі PetriObjModel, яка просуває час
 * до найближчої події і рахує середні за часом значення маркерів і зайнятості переходів.
 */
final class Sim {
    final PetriNet net;
    double time;


    Sim(PetriNet net) {
        this.net = net;
    }


    /** Запам'ятовувати моменти входу і виходу маркерів переходу (щоб рахувати запуски). */
    Sim track(String... transitions) {
        for (String name : transitions) {
            t(name).setMoments(true);
        }
        return this;
    }


    Sim run(double time) {
        this.time = time;
        ArrayList<PetriSim> objects = new ArrayList<>(List.of(new PetriSim(net, new StateTime())));
        PetriObjModel model = new PetriObjModel(objects);
        model.setIsProtokol(false);
        model.go(time);
        return this;
    }


    PetriP p(String name) {
        for (PetriP place : net.getListP()) {
            if (place.getName().equals(name)) {
                return place;
            }
        }
        throw new IllegalArgumentException("немає позиції " + name);
    }


    PetriT t(String name) {
        for (PetriT transition : net.getListT()) {
            if (transition.getName().equals(name)) {
                return transition;
            }
        }
        throw new IllegalArgumentException("немає переходу " + name);
    }


    /** Кількість маркерів у позиції в кінці моделювання. */
    int mark(String place) {
        return p(place).getMark();
    }
```

```
    /** Середня за часом кількість маркерів у позиції. */
    double mean(String place) {
        return p(place).getMean();
    }


    /** Середня за часом кількість маркерів, що одночасно перебувають у переході. */
    double busy(String transition) {
        return t(transition).getMean();
    }


    /** Скільки разів маркери вийшли з переходу (потрібен track). */
    int fired(String transition) {
        return t(transition).getOutMoments().size();
    }


    /** Моменти виходу маркерів з переходу (потрібен track). */
    List<Double> exits(String transition) {
        return t(transition).getOutMoments();
    }
}
```

Завдання 1

Мережа Петрі

Мережу наведено на рисунку 1. Позиція «генератор» з переходом «надходження» (детермінована затримка 0,25 хв) подає деталь у позицію «біля пристрою 1». Для кожного пристрою і позиція «пристрій і вільний» з одним маркером задає стан пристрою. Деталь біля пристрою може запустити два конфліктні переходи: «обробка і» (пріоритет 1, умова — пристрій вільний) або «конвеєр і–(і+1)» (пріоритет 0, затримка 1 хв). Тому деталь обробляється, якщо пристрій вільний, і їде далі, якщо зайнятий. Від п’ятого пристрою деталь повертається до першого переходом «повернення до 1» із затримкою 5 хв. Переходи конвеєра багатоканальні: на стрічці одночасно може бути багато деталей.

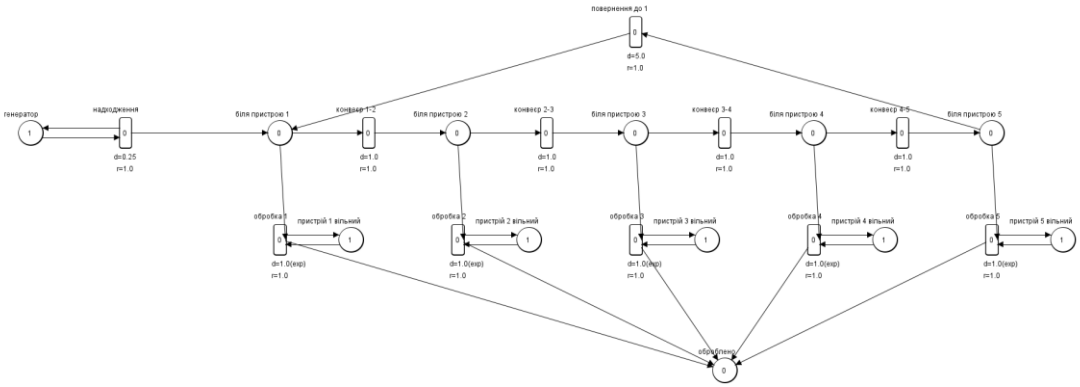


Рисунок 1 – Мережа Петрі конвеєрної системи

Таблиця 1 – Переходи мережі конвеєра

Перехід	Затримка, хв	Пріоритет	Подія
надходження	0,25 (детермінована)	0	деталь надходить до першого пристрою
обробка і, і = 1…5	експоненційна, 1	1	пристрій вільний — деталь обробляється
конвеєр і–(і+1), і = 1…4	1 (детермінована)	0	пристрій зайнятий — деталь їде далі
повернення до 1	5 (детермінована)	0	усі пристрої були зайняті

Лістинг файлів

Файл src\Task1Conveyor.java:

```
/** Завдання 1: конвеєрна система з п'яти пристроїв (час у хвилинах). */
public class Task1Conveyor {
    static final double TIME = 10_000, SERVICE = 1, STEP = 1, BACK = 5;


    public static void main(String[] args) throws Exception {
        Sim s = new Sim(Nets.CreateNetConveyor(0.25))
                .track("надходження", "конвеєр 1-2", "конвеєр 2-3", "конвеєр 3-4", "конвеєр 4-5",
                        "повернення до 1")
                .run(TIME);
        int came = s.fired("надходження");
        int done = s.mark("оброблено");
        double busy = 0, inSystem = 0;
        System.out.printf("Конвеєр, T = %.0f хв, надходить 4 деталі/хв%n", TIME);
        System.out.println("Пристрій | завантаження");
        for (int i = 1; i <= 5; i++) {
            double load = 1 - s.mean("пристрій " + i + " вільний");
            busy += load;
            inSystem += s.busy("обробка " + i);
            System.out.printf("%8d | %12.4f%n", i, load);
        }
        int passes = 0;
        for (int i = 1; i <= 4; i++) {
            passes += s.fired("конвеєр " + i + "-" + (i + 1));
            inSystem += s.busy("конвеєр " + i + "-" + (i + 1));
        }
        int returns = s.fired("повернення до 1");
        inSystem += s.busy("повернення до 1");
        double throughput = done / TIME;
        System.out.printf("Надійшло деталей %d, оброблено %d%n", came, done);
        System.out.printf("Середня кількість зайнятих пристроїв: %.4f (λ·тобр = 4)%n", busy);
        System.out.printf("Середня кількість деталей у системі: %.4f%n", inSystem);
        System.out.printf("Середній час перебування деталі в системі: %.4f хв (формула Літтла)%n",
                inSystem / throughput);
        System.out.printf("  на деталь: %.4f переміщень між пристроями, "
                + "%.4f повернень до першого;%n", (double) passes / done, (double) returns / done);
        System.out.printf("  обробка + переміщення: %.4f хв%n",
                SERVICE + STEP * passes / done + BACK * returns / done);
    }
}
```

Результат роботи

```
Конвеєр, T = 10000 хв, надходить 4 деталі/хв
Пристрій | завантаження
       1 |       0.8868
       2 |       0.8546
       3 |       0.8113
       4 |       0.7501
       5 |       0.6799
Надійшло деталей 40000, оброблено 39967
Середня кількість зайнятих пристроїв: 3.9827 (λ·тобр = 4)
Середня кількість деталей у системі: 34.9516
Середній час перебування деталі в системі: 8.7451 хв (формула Літтла)
  на деталь: 4.4883 переміщень між пристроями, 0.6519 повернень до першого;
  обробка + переміщення: 8.7476 хв
```

Висновок

За 10000 хв надійшло 40000 деталей, оброблено 39967. Завантаження пристроїв спадає вздовж конвеєра: 0,89, 0,85, 0,81, 0,75, 0,68, бо перший пристрій отримує всі деталі, а кожний наступний — лише ті, що не потрапили на

попередні. У середньому зайнято 3,98 пристрою, що збігається з добутком інтенсивності надходження на середній час обробки $4 \cdot 1 = 4$: кожна деталь рано чи пізно обробляється одним пристроєм.

Деталь перебуває в системі в середньому 8,75 хв (у системі одночасно 35,0 деталі). Цей час складається з обробки (1 хв) і переміщень: у середньому 4,49 переходу між пристроями по 1 хв і 0,65 повернення до першого по 5 хв, що дає 8,75 хв — той самий результат іншим способом. Навантаження $\rho = 4/5 = 0{,}8$ велике, тому деталь у середньому кілька разів проходить конвеєр, перш ніж знайде вільний пристрій.

Завдання 2

Формалізація

Пункт прибуття, верстати і склад розташовані вздовж однієї лінії, тому час переміщення між будь-якими двома точками дорівнює сумі часів між сусідніми: від пункту прибуття до другого верстата — 13 с, до складу — 18 с. Прийнято такі припущення.

- Робот бере деталь, лише якщо на наступному верстаті є вільне місце, і бронює його в момент виклику; інакше робот міг би привезти деталь на заповнений верстат і заблокувати систему.
- Оброблена деталь займає місце на верстаті, доки робот її не захопить; місце звільняється в кінці захоплення.
- Переміщення з деталлю і її вивільнення подано одним переходом із рівномірною затримкою d + (8±1) с, бо сума сталої і рівномірної величини — рівномірна величина.
- У варіанті 1 робот після вивільнення деталі холостим ходом повертається на початок свого шляху. У варіанті 2 робот залишається там, де вивільнив деталь, а до деталі, що чекає, їде найближчий вільний робот: переходи «робот X → Y» мають затримку, що дорівнює відстані, і пріоритет 100 − відстань.

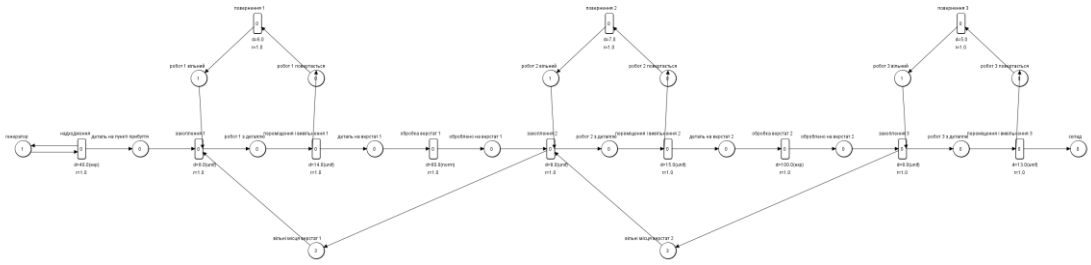


Рисунок 2 – Мережа Петрі ГВС, варіант 1: по одному роботу на кожному шляху

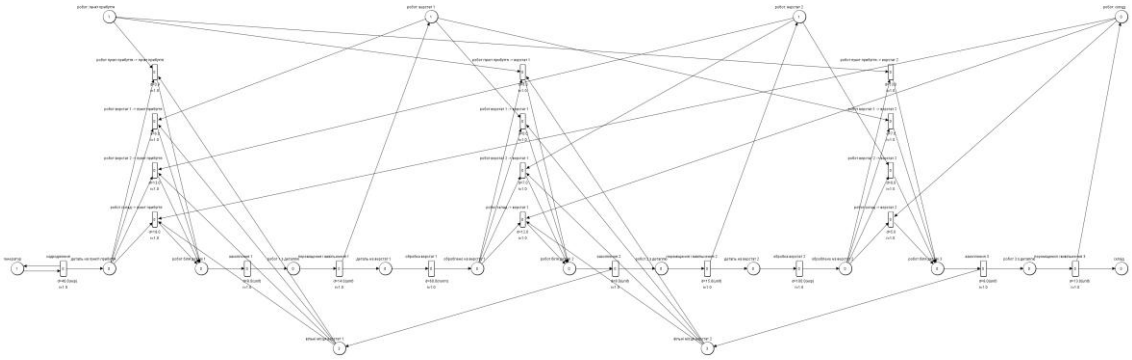


Рисунок 3 – Мережа Петрі ГВС, варіант 2: кожний робот на кожному шляху

Таблиця 2 – Переходи мереж гнучкої виробничої системи

Перехід	Затримка, с	Пріоритет
надходження	експоненційна, 40	0
обробка верстат 1 / верстат 2	нормальна (60; 10) / експоненційна, 100	0
захоплення k	рівномірна, 8 ± 1	0
переміщення і вивільнення k	рівномірна, d + 8 ± 1 (d = 6, 7, 5)	0
повернення k (варіант 1)	d (детермінована)	0
робот X → Y (варіант 2)	відстань між X і Y	100 − відстань

Лістинг файлів

Файл src\Task2Robots.java:

```
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;


/** Завдання 2: роботизована гнучка виробнича система, два способи закріплення роботів (с). */
public class Task2Robots {
    static final double TIME = 360_000; // 100 годин


    record Result(double perHour, double queue, double robots, double[] occupied,
                  double[] processing, double timeInSystem, int came, int stored) {
    }


    static Result run(boolean shared, double arrival) throws Exception {
        PetriNet net = shared ? Nets.CreateNetRobotsShared(arrival)
                : Nets.CreateNetRobotsDedicated(arrival);
        Sim s = new Sim(net).track("надходження").run(TIME);
        double robotsFree = 0, inSystem = 0;
        for (PetriP p : net.getListP()) {
            String n = p.getName();
            if (n.startsWith("робот: ") || n.endsWith(" вільний")) {
                robotsFree += p.getMean();
            }
            if (n.startsWith("деталь на") || n.startsWith("оброблено на")
                    || n.contains("з деталлю") || n.startsWith("робот біля деталі")) {
                inSystem += p.getMean(); // деталь чекає, лежить на верстаті або в руках робота
            }
        }
        for (PetriT t : net.getListT()) {
            String n = t.getName();
            if (n.startsWith("захоплення") || n.startsWith("переміщення")
                    || n.startsWith("обробка") || n.contains(" -> ")) {
```

```
                inSystem += t.getMean(); // деталь обробляється або до неї вже їде робот
            }
        }
        int stored = s.mark("склад");
        double throughput = stored / TIME;
        double[] occupied = {3 - s.mean("вільні місця верстат 1"),
            3 - s.mean("вільні місця верстат 2")};
        double[] processing = {s.busy("обробка верстат 1"), s.busy("обробка верстат 2")};
        return new Result(throughput * 3600, s.mean("деталь на пункті прибуття"), 3 - robotsFree,
                occupied, processing, inSystem / throughput, s.fired("надходження"), stored);
    }


    static void print(String title, Result r) {
        System.out.println(title);
        System.out.printf("  надійшло %d деталей, на складі %d%n", r.came, r.stored);
        System.out.printf("  пропускна здатність: %.2f деталі/год (надходить 90 деталей/год)%n",
                r.perHour);
        System.out.printf("  середня черга на пункті прибуття: %.2f; зайнятих роботів: %.3f з 3%n",
                r.queue, r.robots);
        for (int m = 0; m < 2; m++) {
            System.out.printf("  верстат %d: зайнято місць %.3f з 3, з них обробка %.3f%n",
                    m + 1, r.occupied[m], r.processing[m]);
        }
    }


    public static void main(String[] args) throws Exception {
        System.out.printf("Гнучка виробнича система, T = %.0f с (100 год)%n", TIME);
        print("Варіант 1: по одному роботу на кожному шляху", run(false, 40));
        print("Варіант 2: кожен робот на кожному шляху, їде найближчий", run(true, 40));
        System.out.println("\nПропускна здатність при різному інтервалі надходження, деталей/год:");
        System.out.println(" інтервал, с | надходить | варіант 1 | варіант 2 "
                + "| час у системі, с: варіант 1 | варіант 2");
        for (double arrival : new double[]{60, 50, 45, 40, 30, 20}) {
            Result a = run(false, arrival), b = run(true, arrival);
            System.out.printf(" %11.0f | %9.1f | %9.2f | %9.2f | %26.0f | %9.0f%n", arrival,
                    3600 / arrival, a.perHour, b.perHour, a.timeInSystem, b.timeInSystem);
        }
    }
}
```

Результат роботи

```
Гнучка виробнича система, T = 360000 с (100 год)
Варіант 1: по одному роботу на кожному шляху
  надійшло 8930 деталей, на складі 7622
  пропускна здатність: 76.22 деталі/год (надходить 90 деталей/год)
  середня черга на пункті прибуття: 682.82; зайнятих роботів: 1.779 з 3
  верстат 1: зайнято місць 2.997 з 3, з них обробка 1.273
  верстат 2: зайнято місць 2.896 з 3, з них обробка 2.101
Варіант 2: кожен робот на кожному шляху, їде найближчий
  надійшло 8995 деталей, на складі 7275
  пропускна здатність: 72.75 деталі/год (надходить 90 деталей/год)
  середня черга на пункті прибуття: 872.21; зайнятих роботів: 1.730 з 3
  верстат 1: зайнято місць 2.950 з 3, з них обробка 1.214
  верстат 2: зайнято місць 2.776 з 3, з них обробка 1.990


Пропускна здатність при різному інтервалі надходження, деталей/год:
 інтервал, с | надходить | варіант 1 | варіант 2 | час у системі, с: варіант 1 | варіант 2
          60 |      60.0 |     59.17 |     60.18 |                          344 |       378
          50 |      72.0 |     72.19 |     70.46 |                          719 |      1154
          45 |      80.0 |     75.86 |     72.17 |                        12160 |     17960
          40 |      90.0 |     75.60 |     73.03 |                        31631 |     41770
          30 |     120.0 |     75.24 |     70.88 |                       100520 |    116591
          20 |     180.0 |     75.60 |     72.09 |                       245581 |    272663
```

Таблиця 3 – Пропускна здатність ГВС при різному інтервалі надходження деталей

Інтервал, с	Надходить, дет./год	Варіант 1, дет./год	Варіант 2, дет./год	Час у системі, с: варіант 1	Варіант 2
60	60,0	59,17	60,18	344	378
50	72,0	72,19	70,46	719	1154
45	80,0	75,86	72,17	12160	17960
40	90,0	75,60	73,03	31631	41770
30	120,0	75,24	70,88	100520	116591
20	180,0	75,60	72,09	245581	272663

Висновок

При надходженні деталей раз на 40 с (90 деталей/год) система перевантажена в обох варіантах: черга на пункті прибуття необмежено зростає, а на склад надходить 76,2 деталі/год у варіанті 1 і 72,8 — у варіанті 2. «Вузьке місце» — верстати: зайнято 3,00 і 2,90 місця з трьох, але обробка триває лише на 1,27 і 2,10 з них; решту часу місце зайняте деталлю, що чекає на робота або на місце на наступному верстаті, чи забронюване під деталь, яку везуть. Роботи при цьому зайняті менш ніж на дві третини (1,78 з 3).

Пропускна здатність — це найбільший потік, який система здатна обслужити, тому вона визначається на перевантаженій системі (інтервал 45 с і менше). За таблицею 3 закріплення по одному роботу на кожному шляху стабільно дає близько 75–76 деталей/год, а використання будь-якого робота на будь-якому шляху — 71–73. Причина в тому, що у варіанті 2 робот залишається там, де вивільнив деталь — біля верстата або складу, — і лише після появи нової деталі їде до неї холостим ходом; увесь цей час місце на наступному верстаті забронюване. У варіанті 1 робот повертається на свій шлях одразу після вивільнення деталі і зустрічає наступну деталь уже на місці.

При малому навантаженні (інтервал 60 с) обидва варіанти обслуговують увесь потік, і час перебування деталі майже однаковий; при інтервалі 50 с і менше у варіанті 2 він більший. Отже, найкращим з точки зору пропускної здатності є варіант 1 — закріплення по одному роботу за кожним шляхом.

Завдання 3

Формалізація

Слова «він їде у зворотному напрямку тим же чином» трактуються так: на обох кінцевих зупинках пасажири підходять за тим самим законом, і на кожній зупинці мікроавтобус чекає, доки заповняться всі місця. На початку дня автобус A стоїть на зупинці 1, автобус B — на зупинці 2; плата за проїзд сплачується при посадці.

Для кожної зупинки k мережа (рисунок 4) містить генератор пасажирів, позицію «пасажир прийшов», конфліктні переходи «відмова на k» (пріоритет 2, інформаційна дуга з кратністю 30 від позиції черги: спрацьовує лише тоді, коли в черзі вже 30 осіб) і «у чергу k» (пріоритет 1). Для кожного автобуса — позиції «вільні місця» (n маркерів), «пасажири в салоні» і «автобус на зупинці k». Посадка в A має пріоритет 4, посадка в B — 3, тому пасажир сідає в B лише тоді, коли біля зупинки немає автобуса A з вільними місцями; наявність автобуса на зупинці перевіряється інформаційною дугою. Перехід «рейс» (пріоритет 5) забирає n пасажирів, коли всі місця зайняті, а «висадка» повертає автобус на іншу зупинку і n вільних місць.

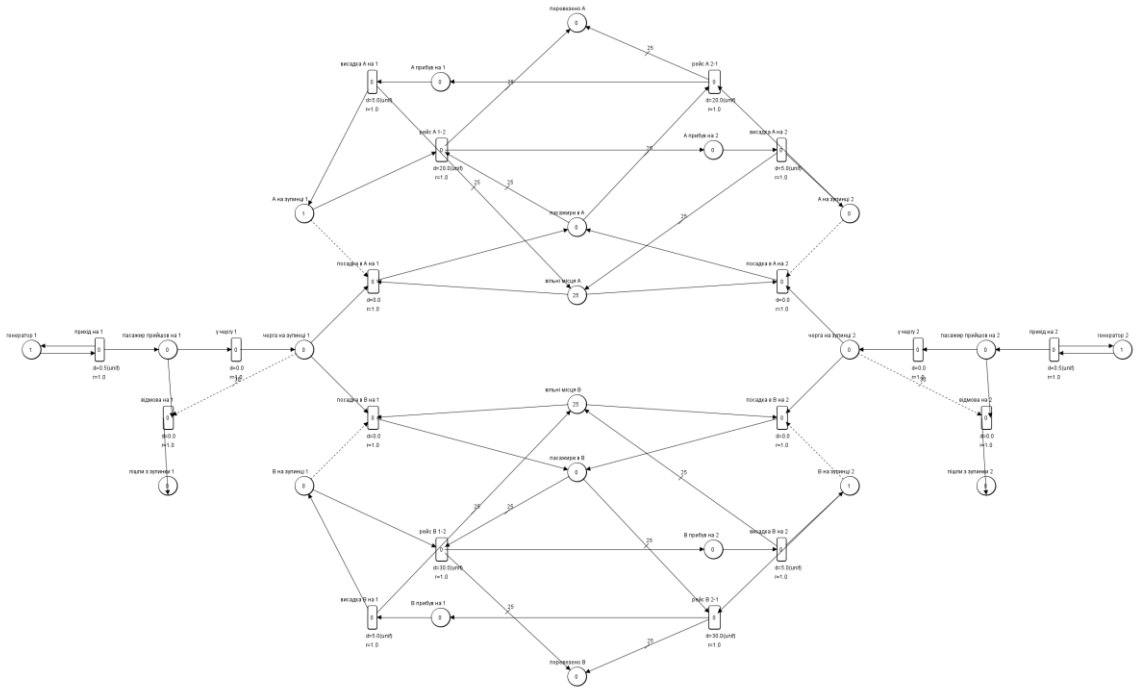


Рисунок 4 – Мережа Петрі маршруту з двома мікроавтобусами (n = 25)

Таблиця 4 – Переходи мережі маршруту

Перехід	Затримка, хв	Пріоритет	Дуги
прихід на k	рівномірна, 0,5 ± 0,2	0	—
відмова на k	0	2	інформаційна з черги, кратність 30
у чергу k	0	1	—
посадка в A / B на k	0	4 / 3	інформаційна з «A (B) на зупинці k»
рейс A / B	рівномірна, 20 ± 5 / 30 ± 5	5	з «пасажири в A (B)» кратністю n
висадка A / B	рівномірна, 5 ± 1	0	у «вільні місця» кратністю n

Лістинг файлів

Файл src\Task3Minibus.java:

```
/** Завдання 3: два мікроавтобуси на приміському маршруті (час у хвилинах). */
public class Task3Minibus {
    static final double DAY = 600; // 10 годин роботи
    static final int DAYS = 40, FARE = 20;
    static final String[] BOARDING = {"посадка в A на 1", "посадка в A на 2", "посадка в B на 1",
        "посадка в B на 2"};


    record Day(double queueWait, double inBus, int boarded, int lost, int revenue) {
    }


    static Day day(int seats) throws Exception {
        Sim s = new Sim(Nets.CreateNetMinibus(seats)).track(BOARDING).run(DAY);
        int boarded = 0; // сіли в мікроавтобус і заплатили за проїзд
```

```
        for (String t : BOARDING) {
            boarded += s.fired(t);
        }
        int lost = s.mark("пішли з зупинки 1") + s.mark("пішли з зупинки 2");
        double queue = s.mean("черга на зупинці 1") + s.mean("черга на зупинці 2");
        double seated = s.mean("пасажири в A") + s.mean("пасажири в B");
        // середній час очікування = сумарне очікування (Σ M·Δt) / кількість пасажирів, що сіли
        return new Day(queue * DAY / boarded, seated * DAY / boarded, boarded, lost,
                FARE * (boarded - lost));
    }


    public static void main(String[] args) throws Exception {
        System.out.printf("Маршрутки: %d днів по %.0f хв для кожного n%n", DAYS, DAY);
        System.out.println("  n | очікування в черзі, хв | у салоні, хв | сіли | пішли "
                + "| оплата, грн | недоотримано, грн | виручка, грн");
        int best = 0;
        double bestWait = Double.MAX_VALUE;
        for (int seats = 5; seats <= 25; seats += 5) {
            double wait = 0, inBus = 0, boarded = 0, lost = 0, revenue = 0, revenue2 = 0;
            for (int d = 0; d < DAYS; d++) {
                Day r = day(seats);
                wait += r.queueWait / DAYS;
                inBus += r.inBus / DAYS;
                boarded += (double) r.boarded / DAYS;
                lost += (double) r.lost / DAYS;
                revenue += (double) r.revenue / DAYS;
                revenue2 += (double) r.revenue * r.revenue / DAYS;
            }
            double error = Math.sqrt((revenue2 - revenue * revenue) / (DAYS - 1));
            System.out.printf(" %2d | %22.2f | %12.3f | %4.0f | %5.0f | %11.0f | %17.0f "
                    + "| %7.0f ± %.0f%n", seats, wait, inBus, boarded, lost, FARE * boarded,
                    FARE * lost, revenue, error);
            if (wait < bestWait) {
                bestWait = wait;
                best = seats;
            }
        }
        System.out.printf("Найменше очікування в черзі — при n = %d: %.2f хв%n", best, bestWait);
        System.out.println("Виручка = плата всіх, хто сів, − недоотримане від тих, хто пішов; ± — "
                + "стандартна похибка середнього за днями");
    }
}
```

Результат роботи

```
Маршрутки: 40 днів по 600 хв для кожного n
  n | очікування в черзі, хв | у салоні, хв | сіли | пішли | оплата, грн | недоотримано, грн | виручка, грн
  5 |                 167.31 |        0.047 |  210 |  2127 |        4200 |             42539 |  -38339 ± 44
 10 |                  81.53 |        0.108 |  419 |  1923 |        8380 |             38453 |  -30074 ± 82
 15 |                  52.24 |        0.167 |  624 |  1721 |       12473 |             34419 |  -21947 ± 73
 20 |                  36.84 |        0.387 |  825 |  1522 |       16490 |             30430 |  -13940 ± 87
 25 |                  27.75 |        1.041 |  997 |  1355 |       19941 |             27092 |   -7151 ± 134
Найменше очікування в черзі — при n = 25: 27.75 хв
Виручка = плата всіх, хто сів, − недоотримане від тих, хто пішов; ± — стандартна похибка середнього за днями
```

Таблиця 5 – Характеристики маршруту за день (середні за 40 днів)

n	Очікування в черзі, хв	Сіли	Пішли	Оплата, грн	Недоотримано, грн	Виручка, грн
5	167,31	210	2127	4200	42539	-38339 ± 44
10	81,53	419	1923	8380	38453	-30074 ± 82
15	52,24	624	1721	12473	34419	-21947 ± 73
20	36,84	825	1522	16490	30430	-13940 ± 87
25	27,75	997	1355	19941	27092	-7151 ± 134

Висновок

Маршрут сильно перевантажений: за день на дві зупинки приходить близько 2400 пасажирів, а два мікроавтобуси навіть при n = 25 можуть забрати лише близько 997. Черга майже весь час заповнена до 30 осіб, і 1355 пасажирів за день ідуть до іншого маршруту.

Час очікування в черзі зменшується зі збільшенням кількості місць: кожний рейс забирає більше пасажирів, а тривалість рейсу від n не залежить. Найменше очікування — 27,75 хв — при найбільшому допустимому n = 25; час очікування в салоні до відправлення при цьому лише 1,04 хв, бо автобус заповнюється одразу з черги.

Плата за проїзд при n = 25 становить 19941 грн за день, а недоотримано від пасажирів, які пішли, — 27092 грн, тож виручка з урахуванням втрат від’ємна: -7151 грн за день. Двох мікроавтобусів недостатньо для такого потоку пасажирів; щоб маршрут був прибутковим, потрібно більше автобусів.

Завдання 4

Мережа Петрі

Мережу наведено на рисунку 5. Перехід «попит» (експоненційна затримка 0,2 тижня) створює покупця. Якщо холодильник є, спрацьовує «продаж» (пріоритет 2). Якщо його немає, конфліктують переходи «інший магазин» (імовірність 0,8) і «повторне замовлення» (0,2); обидва мають пріоритет 1, щоб у розіграші не брав участі перехід «попит» (див. підхід до моделювання). Невдоволений попит задовольняється переходом «видача відкладених» (пріоритет 3), щойно в запасі з’являються холодильники.

Правило замовлення залежить від поточного стану запасу, який у мережі не зберігається в одній позиції. Тому введено позицію «нестача до рівня», кількість маркерів у якій завжди дорівнює 72 мінус поточний стан запасу: продаж і повторне замовлення додають до неї по маркеру (стан запасу зменшується на 1), видача відкладених її не змінює (зменшуються і запас, і невдоволений попит), а розміщення замовлення переносить з неї всі маркери в замовлені. Умова «стан запасу ≤ 18» рівносильна умові «нестача ≥ 54», яку перевіряє інформаційна дуга з кратністю 54.

Перехід «перегляд» (детермінована затримка 4 тижні) раз на чотири тижні подає маркер у позицію «перегляд запасу». Якщо нестача не менша за 54, спрацьовує «замовити» (пріоритет 2), інакше — «не замовляти». Під час розміщення замовлення перехід «замовлення одиниці» (пріоритет 3) переносить маркери з позиції нестачі в позицію «замовлено», доки вона не спорожніє, а потім «кінець замовлення» завершує його. Багатоканальний перехід «доставка» з детермінованою затримкою 3 тижні доставляє всі холодильники партії одночасно. Розмір замовлення дорівнює нестачі, тобто доводить стан запасу до 72.

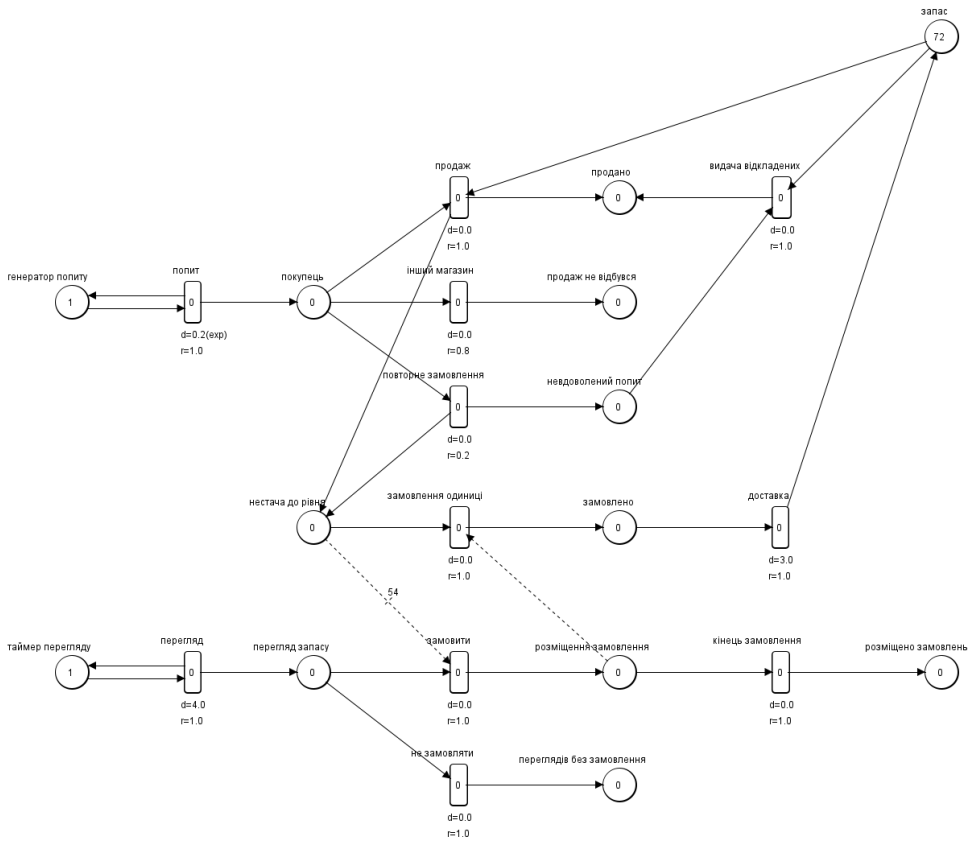


Рисунок 5 – Мережа Петрі системи управління запасами холодильників

Лістинг файлів

Файл src\Task4Fridges.java:

```
import java.util.List;


/** Завдання 4: управління запасами холодильників (час у тижнях). */
public class Task4Fridges {
    static final double TIME = 20_000;


    public static void main(String[] args) throws Exception {
        Sim s = new Sim(Nets.CreateNetFridges(72, 18, 4))
                .track("попит", "інший магазин", "доставка").run(TIME);
        int customers = s.fired("попит"), sold = s.mark("продано");
        int lost = s.mark("продаж не відбувся"), backlog = s.mark("невдоволений попит");
        List<Double> moments = s.exits("інший магазин");
        double between = (moments.getLast() - moments.getFirst()) / (moments.size() - 1);
        System.out.printf("Запаси холодильників, T = %.0f тижнів%n", TIME);
        System.out.printf("Покупців %d: продано %d, продаж не відбувся %d, очікують %d%n",
                customers, sold, lost, backlog);
        System.out.printf("Середня кількість холодильників у запасі: %.4f%n", s.mean("запас"));
        System.out.printf("Середній час між продажами, що не відбулися: %.4f тижня "
                + "(T / кількість = %.4f)%n", between, TIME / lost);
        System.out.printf("Частка покупців, що пішли в інший магазин: %.4f%n",
                (double) lost / customers);
        System.out.printf("Середній невдоволений попит: %.4f%n", s.mean("невдоволений попит"));
        int orders = s.mark("розміщено замовлень");
        int skipped = s.mark("переглядів без замовлення");
        System.out.printf("Переглядів запасу %d, розміщено замовлень %d, "
                + "середній розмір замовлення %.2f%n", orders + skipped, orders,
                (double) s.fired("доставка") / orders);
        System.out.printf("Баланс покупців: %d = %d + %d + %d: %s%n", customers, sold, lost,
                backlog, customers == sold + lost + backlog ? "так" : "НІ");
```

```
    }
}
```

Результат роботи

```
Запаси холодильників, T = 20000 тижнів
Покупців 100065: продано 91534, продаж не відбувся 8531, очікують 0
Середня кількість холодильників у запасі: 28.8462
Середній час між продажами, що не відбулися: 2.3429 тижня (T / кількість = 2.3444)
Частка покупців, що пішли в інший магазин: 0.0853
Середній невдоволений попит: 0.1310
Переглядів запасу 4999, розміщено замовлень 1443, середній розмір замовлення 63.42
Баланс покупців: 100065 = 91534 + 8531 + 0: так
```

Висновок

За 20000 тижнів надійшло 100065 покупців: продано 91534 холодильників, 8531 продажів не відбулося; баланс покупців виконується. Середня кількість холодильників у запасі — 28,85, середній час між продажами, що не відбулися, — 2,34 тижня (близько 16 днів); в інший магазин іде 8,5 % покупців.

Замовлення розміщується не на кожному перегляді: з 4999 переглядів замовлення зроблено 1443 разів, у середньому на 63,4 холодильника. За 4 тижні продається близько 20 холодильників, тому від повного запасу 72 до точки замовлення 18 запас знижується приблизно за 11–12 тижнів, і замовлення робиться приблизно на кожному третьому-четвертому перегляді. Між розміщенням замовлення і доставкою минає 3 тижні, протягом яких запас нерідко вичерпується — саме тоді й відбуваються втрачені продажі.

Висновок

У ході виконання комп’ютерного практикуму чотири дискретно-подійні системи формалізовано стохастичними мережами Петрі, побудованими в редакторі PetriObjModelPaint, і їх характеристики визначено імітацією алгоритмом цієї бібліотеки.

1. Конвеєр: завантаження пристроїв від 0,89 до 0,68, у середньому зайнято 3,98 пристрою (теоретично 4), деталь перебуває в системі 8,75 хв.
2. Гнучка виробнича система при надходженні раз на 40 с перевантажена; закріплення по одному роботу за кожним шляхом дає більшу пропускну здатність (76,2 проти 72,8 деталі/год).
3. Маршрут мікроавтобусів перевантажений; найменше очікування в черзі (27,8 хв) — при n = 25, виручка з урахуванням втрачених пасажирів від’ємна.
4. Запаси холодильників: середній запас 28,85, середній час між продажами, що не відбулися, 2,34 тижня.

Мережі Петрі дали змогу описати не лише процеси обслуговування, а й керування: бронювання місць, вибір найближчого робота, правило замовлення з порогом. Для цього використано пріоритети і ймовірності конфліктних переходів, багатоканальні переходи, інформаційні дуги та кратні дуги.

Посилання

1. Комп’ютерний практикум 5 «Формалізація дискретно-подійних систем стохастичною мережею Петрі»: методичні вказівки з дисципліни «Комп’ютерне моделювання дискретно-подійних систем». – КПІ ім. Ігоря Сікорського, 2026.
2. Стеценко І. В. Моделювання систем: навч. посіб. – Черкаси: ЧДТУ, 2010. – 399 с.
3. PetriObjModelPaint — графічний редактор і симулятор мереж Петрі та Петрі-об’єктних моделей. – URL: https://github.com/StetsenkoInna/PetriObjModelPaint
4. Повний код практичної роботи: https://github.com/ip34is/7-sem/tree/main/CMDES/CMDES-lab5